TABLE I
SUMMARY OF KEY NOTATIONS

Notation	Description
System Parameters	
λ, n, Δ	Security parameter, RLWE dimension, scaling factor
$\mathbb{Z}_q, \mathcal{R}_q, \mathcal{R}_p$	Integer, ciphertext, and plaintext rings
φ, M	Ring isomorphism; prime bounding modulus
χ_s, χ_e	Secret-key and error distributions
Clients and Rounds	
$\mathcal{C}, N, K$	All clients, total count, selected count
$T, t, \mathcal{E}$	Total rounds, current round, local epochs
τ, g_i	Round index; rounds client i missed ($g_i = t - t_{\text{last},i}$)
$\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}$	Active and inactive client sets
$t_{\text{last},i}$	Client i's last active round
Models and Ciphertext States	
$\mathbf{W}^{(t)}, \mathbf{w}_i^{(t)}$	Global model; local model
$G^{(t)}, G_j^{(t)}$	Global/pack-level complete model ciphertexts
$C^{(t)}, C_j^{(t)}$	Global/pack-level aggregated update ciphertexts
$\mathbf{\Delta}^{(t)}$	KGC compensatory perturbation
Sparse Consensus and Voting	
$\mathcal{T}_i^{(t)}, \mathcal{K}^{(t)}$	Local Top-k indices; consensus mask (also the selected index set)
$\mathbf{v}_i^{(t)}, v_{i,j}^{(t)}$	Vote bitmap; binary voting element
$s_j^{(t)}, \theta, \lambda_\theta$	Global score; selection ratio; truncation threshold
Gradients and Residuals	
$\mathbf{g}_i^{(t)}, \tilde{\mathbf{g}}_i^{(t)}, \mathbf{r}_i^{(t)}$	Raw gradient; compensated gradient; pack residual
$\text{pt}_{i,j}^{(t)}, e_{i,j}^{(t)}$	Encoded gradient polynomial; noise polynomial
$\tilde{p}$	Plaintext-modulus noise polynomial
γ_d, γ_s	Per-round and cross-round decay factors
ζ	Local gradient pack-level sparsification ratio
Cryptographic Primitives	
S, s_i	Global decryption key; local encryption key
$\mathbf{y}_{i,j}^{(t)}, \mathbf{z}_i^{(t)}, \hat{\mathbf{v}}_i^{(t)}$	Pairwise vector; perturbation; obfuscated vote
$\text{ct}_{i,j}^{(t)}, \text{ct}_{k,j}^{(t)}$	Local upload and compensatory ciphertexts
Prefetching Mechanism	
R	Prefetch-window length ($t^* = t + R$)
t^*	Target round at end of prefetch window ($t^* = t + R$)
$\mathcal{P}^{(t^*)}, P_i^{(t^*)}$	Presampled client set for target t^*; prefetch start round
$\mathcal{P}^{(t)}$	Current prefetch client set at round t
$\mathcal{Q}$	Queue of latest R aggregated update ciphertexts
$\mathcal{L}_{i,j}$	Prefetch accumulator for client i, pack j
$T_{i,j}$	Missed rounds of client i with pack j
$\mathbf{m}_{\text{pre}}, C_{\text{pre},j}$	Prefetched plaintexts and pack-level ciphertexts
$A_{\text{pre},j}$	Accumulated public-polynomial drift $\sum_{\tau \in T_{i,j}} a^{(\tau)}$
ρ, β	EMA decay and over-commitment factors
$\hat{d}^{(t)}, d^{(t)}$	EMA estimate and measured round duration

I. EFFICIENT CIPHERTEXT PREFETCHING

A. ECP: Efficient Ciphertext Prefetching Protocol

1) Sparse Ciphertext Prefetching: The ECP module synchronizes a stale client before it rejoins by prefetching sparse pack-level aggregated update ciphertexts during its idle rounds. Its core idea is *pack-level sparse prefetching driven by the global consensus mask* $\mathcal{K}^{(\tau)}$: instead of downloading the full global ciphertext $G^{(t)}$ or a separate KGC compensatory payload, the client fetches only the selected consensus packs $\{C_j^{(\tau)}\}_{j \in \mathcal{K}^{(\tau)}}$, accumulates them homomorphically, and decrypts once at rejoining. The complete ciphertext $G^{(t)}$ is a server-side authoritative state used for evaluation and server-side state maintenance; the ECP download path never transmits $G^{(t)}$ to clients. Following the MK-CKKS key-generation model, the global decryption key S is distributed to clients, who are assumed to protect S and not to collude to expose it. Algorithm 1 formalizes the download module.

Prepare Phase. At round t, the server selects active clients $\mathcal{S}^{(t)}$, inactive clients $\bar{\mathcal{S}}^{(t)}$, and a prefetch set $\mathcal{P}^{(t^*)}$ for target round $t^* = t + R$, where R is the prefetch horizon. Each client $i \in \mathcal{P}^{(t^*)}$ starts prefetching at round $P_i^{(t^*)} \in \{t, \ldots, t^*-1\}$ based on its bandwidth and availability. The server maintains a sliding queue $\mathcal{Q}$ of length R, which stores the most recent R aggregated update ciphertexts $C^{(\tau)} = \{C_j^{(\tau)}\}_{j \in \mathcal{K}^{(\tau)}}$. Here R is the lookback length of $\mathcal{Q}$ and bounds both the server's historical storage and the maximum recoverable absence length. Because KGC compensatory ciphertexts are already embedded in $C^{(\tau)}$ by ESP-AGG, no separate compensatory payload is stored. The server also advances the global model ciphertext as $G^{(t+1)} = G^{(t)} - \eta C^{(t)}$.

Prefetch Phase. At round t, each client $c_i \in \mathcal{P}^{(t)}$ invokes ECP-DOWNLOAD to scan $\mathcal{Q}$. It downloads pack-level ciphertext $C_j^{(\tau)}$ only when (i) it missed round τ ($t_{\text{last},i} < \tau$), (ii) the update has been aggregated ($\tau \le t$), and (iii) the pack is in the consensus mask ($j \in \mathcal{K}^{(\tau)}$). Downloaded ciphertexts are stored in local accumulator $\mathcal{L}_{i,j}$, keyed by τ to avoid duplicates. Non-consensus packs carry no update and are never transmitted. Undownloaded packs remain in $\mathcal{Q}$ for subsequent idle rounds.

Train Phase. At round t, each selected client $c_i \in \mathcal{S}^{(t)}$ absent for more than one round invokes ECP-SYNC to merge cached ciphertexts and synchronize its local model as $\mathbf{w}_i \leftarrow \mathbf{w}_i - \eta \mathbf{m}_{\text{pre}}$.

ECP is favorable when $g_i \cdot |\mathcal{K}^{(\tau)}| \ll J$, because it replaces a full J-pack download with at most $g_i \cdot |\mathcal{K}^{(\tau)}|$ sparse-pack downloads. If $g_i > R$ or $g_i \cdot |\mathcal{K}^{(\tau)}| \ge J$, the server falls back to sending the full synchronized ciphertext $G^{(t^*)}$.

To handle heterogeneity and dropouts, ECPFL adopts FedFetch's Overcommitment and EMA scheduling [**?**]: the server estimates round duration $\hat{d}^{(t)} = \rho d^{(t-1)} + (1-\rho)\hat{d}^{(t-1)}$ and scales the prefetch pool to $|\mathcal{P}^{(t^*)}| = \lceil \beta K \rceil$. Unlike FedFetch, which prefetches plaintext model deltas, ECP prefetches aggregated ciphertexts and performs homomorphic accumulation followed by one-step decryption, because CKKS ciphertexts cannot be split or inspected without decryption.

2) Client One-Step Decryption: ECPFL avoids per-round decryptions by accumulating missed updates into a single composite ciphertext homomorphically. Each aggregated pack ciphertext has the form

$$C_j^{(\tau)} = \left[a^{(\tau)} S + \tilde{p} E_j^{(\tau)} + \mathbf{m}_j^{(\tau)} \right]_q, \tag{1}$$

where $a^{(\tau)} = \mathsf{PRPG}(B, \tau)$ is the public polynomial, $S = \sum_{i \in \mathcal{C}} s_i$ is the global secret key, and KGC compensatory ciphertexts are already aggregated into $C_j^{(\tau)}$ by ESP-AGG. Algebraic alignment across rounds is thus preserved, and the client can independently add ciphertexts in its local accumulator $\mathcal{L}_{i,j}$.

Algorithm 1: ECP Download Module

Parameter: Prefetch horizon R

Function ECP-QUEUE($t, C^{(t)}, \mathcal{K}^{(t)}, \mathcal{Q}$)
  $\mathcal{Q}$.enqueue$\big((t, \mathcal{K}^{(t)}, C^{(t)})\big)$
  while $\mathcal{Q}$.front().$\tau < t - R + 1$ **do**
    $\mathcal{Q}$.dequeue()

Function ECP-DOWNLOAD($\mathcal{Q}, \{\mathcal{L}_{i,j}\}, t_{\text{last},i}, t$)
  foreach $(\tau, \mathcal{K}^{(\tau)}, C^{(\tau)}) \in \mathcal{Q}$ **do**
    if $t_{\text{last},i} < \tau \le t$ **then**
      foreach $j \in \mathcal{K}^{(\tau)}$ **do**
        if $\tau \notin \{\tau' \mid (\tau', \cdot) \in \mathcal{L}_{i,j}\}$ **then**
          $\mathcal{L}_{i,j} \leftarrow \mathcal{L}_{i,j} \cup \{(\tau, C_j^{(\tau)})\}$

Function ECP-SYNC($\{\mathcal{L}_{i,j}\}, S, \mathbf{w}_i, B, \eta$)
  $\mathbf{m}_{\text{pre}} \leftarrow \mathbf{0}$
  foreach $j \in \{k \mid \mathcal{L}_{i,k} \neq \emptyset\}$ **do**
    $T_{i,j} \leftarrow \{\tau \mid (\tau, C_j^{(\tau)}) \in \mathcal{L}_{i,j}\}$
    $A_{\text{pre},j} \leftarrow \sum_{\tau \in T_{i,j}} \mathsf{PRPG}(B, \tau)$
    $C_{\text{pre},j} \leftarrow \sum_{\tau \in T_{i,j}} C_j^{(\tau)}$
    $\mathbf{m}_{\text{pre},j} \leftarrow \mathsf{HE.Dec}(A_{\text{pre},j}, S, C_{\text{pre},j})$
    Update the j-th pack of $\mathbf{m}_{\text{pre}}$ with $\mathbf{m}_{\text{pre},j}$
  $\{\mathcal{L}_{i,j}\} \leftarrow \emptyset$
  return $\mathbf{w}_i - \eta \mathbf{m}_{\text{pre}}$

For client i and any selected pack j, let $T_{i,j} = \{\tau \mid (\tau, C_j^{(\tau)}) \in \mathcal{L}_{i,j}\}$ denote the missed rounds during which pack j was globally selected. The client computes its local composite ciphertext

$$\begin{aligned} C_{\text{pre},j} &= \sum_{\tau \in T_{i,j}} C_j^{(\tau)} \\ &= \left[\sum_{\tau \in T_{i,j}} \left(a^{(\tau)} S + \tilde{p} E_j^{(\tau)} + \mathbf{m}_j^{(\tau)} \right) \right]_q \\ &= \left[A_{\text{pre},j} S + \tilde{p} E_{\text{pre},j} + \mathbf{m}_{\text{pre},j} \right]_q, \end{aligned} \tag{2}$$

where $A_{\text{pre},j} = \sum_{\tau \in T_{i,j}} a^{(\tau)}$ is the accumulated public-polynomial drift, $E_{\text{pre},j} = \sum_{\tau \in T_{i,j}} E_j^{(\tau)}$ is the composite bounded noise, and $\mathbf{m}_{\text{pre},j}$ is the exact plaintext sum. Notably, $C_{\text{pre},j}$ occupies the same asymptotic space as a single-round ciphertext, regardless of $|T_{i,j}|$.

Unlike interactive distributed decryption protocols that require multi-round share exchanges, ECPFL needs only one decryption per pack. Following the MK-CKKS primitive, the client extracts the plaintext directly from $C_{\text{pre},j}$ using the global decryption key S:

$$\begin{aligned} \mathbf{m}_{\text{pre},j} &\leftarrow \mathsf{HE.Dec}(A_{\text{pre},j}, S, C_{\text{pre},j}) \\ &= \Big[\big[C_{\text{pre},j} - A_{\text{pre},j} S \big]_q \Big]_p \\ &= \Big[\big[A_{\text{pre},j} S + \tilde{p} E_{\text{pre},j} + \mathbf{m}_{\text{pre},j} \big]_q - \big[A_{\text{pre},j} S \big]_q \Big]_p, \end{aligned} \tag{3}$$

where the subtraction of $A_{\text{pre},j} S$ neutralizes the dynamically accumulated public-key component. The client assembles $\{\mathbf{m}_{\text{pre},j}\}$ into the full model update $\mathbf{m}_{\text{pre}}$ and applies it via $\mathbf{w}_i \leftarrow \mathbf{w}_i - \eta \mathbf{m}_{\text{pre}}$. Because ciphertexts are merged homomorphically before decryption, the client performs only one decryption per selected pack, amortizing cryptographic overhead across the entire prefetch horizon.

Algorithm 2: ECPFL Main Procedure

Parameter: $T, \mathcal{E}, \eta, R, \rho, \beta, k, M > N$
Input: Local datasets $\{\mathcal{D}_i\}_{i=1}^N$, initial global model $\mathbf{w}^{(0)}$
Output: Final global model synchronized across clients

// Initialization
KGC generates $\{s_i\}, S = \sum_i s_i, B, \{sd_{i,j}\}_{i<j}$
KGC sends $(s_i, S), \{sd_{i,j}\}_{j \neq i}$ to c_i and B to all clients
Server initializes $\hat{d} \leftarrow d_0$, $G^{(0)}$ from $\mathbf{w}^{(0)}$
foreach $c_i \in \mathcal{C}$ **do**
  $\mathbf{w}_i \leftarrow \mathbf{w}^{(0)}$, $\mathbf{r}_i \leftarrow \mathbf{0}$, $t_{\text{last},i} \leftarrow 0$, $\mathcal{L}_{i,j} \leftarrow \emptyset$ for all j
for $t = 1, 2, \ldots, T$ **do**
  // Server-Side Execution
  Let target prefetch round $t^* = t + R$
  Select active, inactive, and target-round prefetch sets $\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \mathcal{P}^{(t^*)}$
  Assign each $c_i \in \mathcal{P}^{(t^*)}$ a prefetch start round $P_i^{(t^*)} \in \{t, \ldots, t^* - 1\}$
  Derive current prefetch set $\mathcal{P}^{(t)} = \{c_i \mid \exists t' > t : c_i \in \mathcal{P}^{(t')},\ P_i^{(t')} \le t < t'\}$
  Broadcast $\mathcal{S}^{(t)}, \mathcal{P}^{(t)}$ to all clients
  Receive compensatory perturbation $\mathbf{\Delta}^{(t)}$ from KGC
  Receive obfuscated votes $\{\hat{\mathbf{v}}_i^{(t)}\}_{i \in \mathcal{S}^{(t)}}$ from active clients
  $\mathcal{K}^{(t)} \leftarrow$ ESP-CONSENSUS($\{\hat{\mathbf{v}}_i^{(t)}\}_{i \in \mathcal{S}^{(t)}}, \mathbf{\Delta}^{(t)}$)
  Broadcast consensus set $\mathcal{K}^{(t)}$ to active clients
  Receive compensatory ciphertexts $\{\mathsf{ct}_{k,j}^{(t)}\}$ from KGC
  Receive sparse ciphertexts $\{\mathsf{ct}_{i,j}^{(t)}\}$ from active clients
  $\{C_j^{(t)}\} \leftarrow$ ESP-AGG($\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \{\mathsf{ct}_{i,j}^{(t)}\}, \{\mathsf{ct}_{k,j}^{(t)}\}, \mathcal{K}^{(t)}$)
  $G^{(t+1)} \leftarrow G^{(t)} - \eta C^{(t)}$
  ECP-QUEUE($t, C^{(t)}, \mathcal{K}^{(t)}, \mathcal{Q}$)
  // Client-Side Execution
  for *each client* $c_i \in \mathcal{C}$ **do in parallel**
    if $c_i \in \mathcal{S}^{(t)}$ **then**
      $g_i \leftarrow t - t_{\text{last},i}$
      if $g_i > R$ **then**
        Download and decrypt $G^{(t)}$; clear $\{\mathcal{L}_{i,j}\}$
      else if $g_i > 1$ **then**
        $\mathbf{w}_i \leftarrow$ ECP-SYNC($\{\mathcal{L}_{i,j}\}, S, \mathbf{w}_i, B, \eta$)
      $\mathbf{w}_{i,\text{start}} \leftarrow \mathbf{w}_i$; $\mathbf{w}_i \leftarrow \mathsf{SGD}(\mathcal{D}_i, \mathbf{w}_i, \eta, \mathcal{E})$
      $\mathbf{g}_i^{(t)} \leftarrow (\mathbf{w}_{i,\text{start}} - \mathbf{w}_i)/\eta$
      $(\hat{\mathbf{v}}_i^{(t)}, \tilde{\mathbf{g}}_i^{(t)}) \leftarrow$ ESP-VOTE($\mathbf{g}_i^{(t)}, \mathbf{r}_i, k, t$)
      send $\hat{\mathbf{v}}_i^{(t)}$; receive $\mathcal{K}^{(t)}$
      $\{\mathsf{ct}_{i,j}^{(t)}\} \leftarrow$ ESP-ENC($\tilde{\mathbf{g}}_i^{(t)}, \mathcal{K}^{(t)}, g_i, s_i, \mathsf{PRPG}(B, t)$)
      Upload $\{\mathsf{ct}_{i,j}^{(t)}\}$; $t_{\text{last},i} \leftarrow t$
    else if $c_i \in \mathcal{P}^{(t)}$ **then**
      ECP-DOWNLOAD($\mathcal{Q}, \{\mathcal{L}_{i,j}\}, t_{\text{last},i}, t$)
  $d^{(t)} \leftarrow$ measured wall-clock duration of round t

Algorithm 3: ESP Upstream Module

Parameter: $J, k, M, \Delta, \gamma_d, \gamma_s$

Function ESP-VOTE($\mathbf{g}_i^{(t)}, \mathbf{r}_i, k, t$)

  $\tilde{\mathbf{g}}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathbf{r}_i$

  $\mathcal{T}_i \leftarrow \arg\max_{|\mathcal{T}|=k} \sum_{j \in \mathcal{T}} \|\tilde{\mathbf{g}}_{i,j}^{(t)}\|_2^2$

  $\mathbf{v}_i^{(t)} \in \{0,1\}^J$ with $v_{i,j}^{(t)} = 1$ if $j \in \mathcal{T}_i$

  $\mathbf{z}_i^{(t)} \leftarrow [\sum_{j>i} \mathsf{PRG}(sd_{i,j}, t) - \sum_{j<i} \mathsf{PRG}(sd_{j,i}, t)]_M$

  $\hat{\mathbf{v}}_i^{(t)} \leftarrow [\mathbf{v}_i^{(t)} + \mathbf{z}_i^{(t)}]_M$

  return $(\hat{\mathbf{v}}_i^{(t)}, \tilde{\mathbf{g}}_i^{(t)})$

Function ESP-CONSENSUS($\{\hat{\mathbf{v}}_i^{(t)}\}, \mathbf{\Delta}^{(t)}$)

  $\bar{\mathbf{v}}^{(t)} \leftarrow [\sum_{i \in \mathcal{S}^{(t)}} \hat{\mathbf{v}}_i^{(t)} + \mathbf{\Delta}^{(t)}]_M$

  $\mathcal{K}^{(t)} \leftarrow$ Top-k indices of $\bar{\mathbf{v}}^{(t)}$

  return $\mathcal{K}^{(t)}$

Function ESP-ENC($\tilde{\mathbf{g}}_i^{(t)}, \mathcal{K}^{(t)}, g_i, s_i, a^{(t)}$)

  foreach $j \in \mathcal{K}^{(t)}$ **do**

    $\mathsf{pt}_{i,j}^{(t)} \leftarrow \mathsf{CKKS.Encode}(\tilde{\mathbf{g}}_{i,j}^{(t)}, \Delta)$

    $\mathsf{ct}_{i,j}^{(t)} \leftarrow [a^{(t)} s_i + \tilde{pe}_{i,j}^{(t)} + \mathsf{pt}_{i,j}^{(t)}]_q$

  $\mathbf{k}^{(t)} \leftarrow$ binary mask vector for $\mathcal{K}^{(t)}$

  $\mathbf{r}_i \leftarrow \gamma_d \gamma_s^{g_i - 1} [\tilde{\mathbf{g}}_i^{(t)} \odot (\mathbf{1} - \mathbf{k}^{(t)})]$

  return $\{\mathsf{ct}_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}}$

Function ESP-AGG($\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \{\mathsf{ct}_{i,j}^{(t)}\}, \{\mathsf{ct}_{k,j}^{(t)}\}, \mathcal{K}^{(t)}$)

  foreach $j \in \mathcal{K}^{(t)}$ **do**

    $C_j^{(t)} \leftarrow \frac{1}{|\mathcal{S}^{(t)}|} \left[\sum_{i \in \mathcal{S}^{(t)}} \mathsf{ct}_{i,j}^{(t)} + \sum_{k \in \bar{\mathcal{S}}^{(t)}} \mathsf{ct}_{k,j}^{(t)}\right]_q$

  return $\{C_j^{(t)}\}_{j \in \mathcal{K}^{(t)}}$